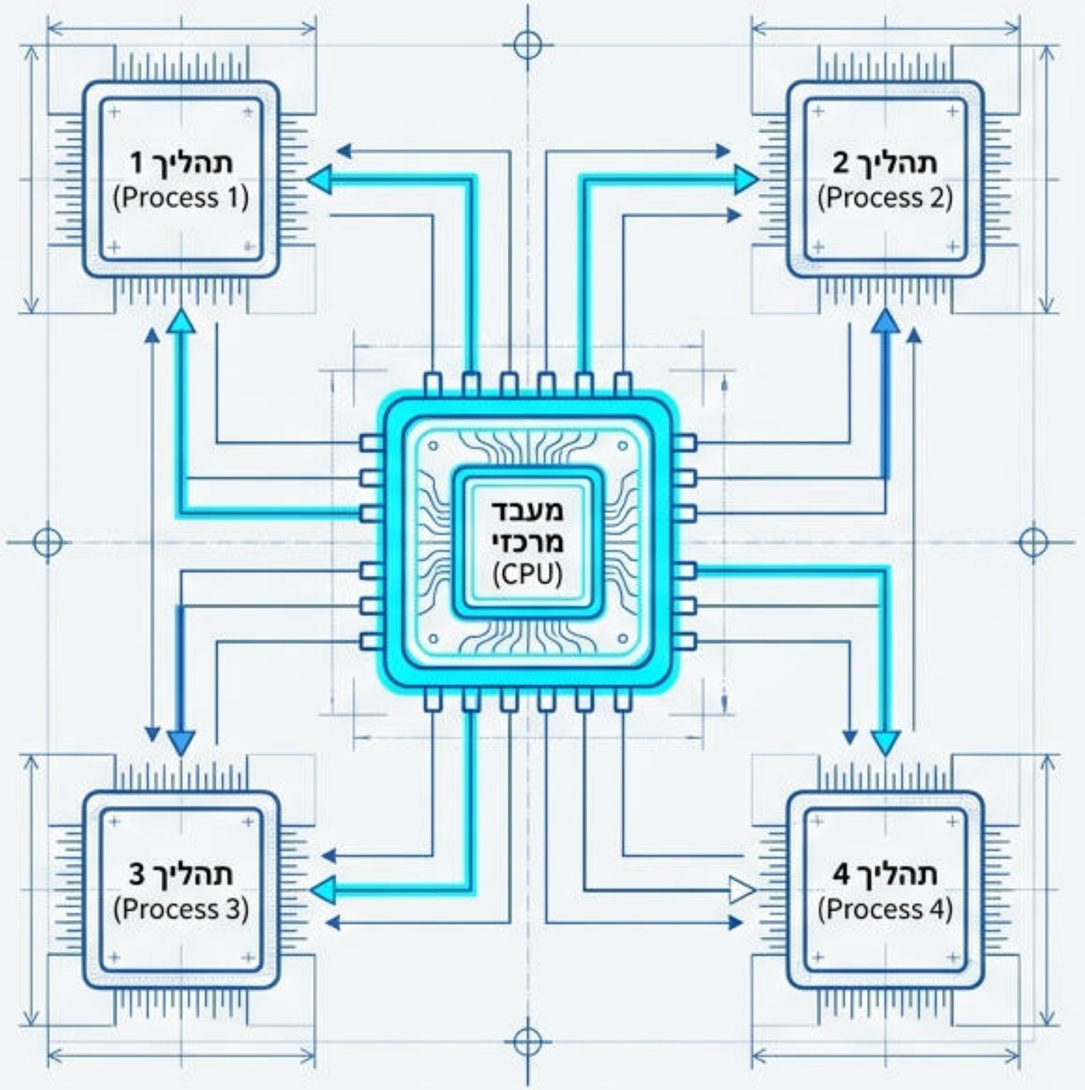
מערכות הפעלה: מעבד, תהליכים וסנכרון

מדריך למידה מקיף לארכיטקטורת המערכת



1. השחקנים: חומרה, תהליכים ותהליכונים.

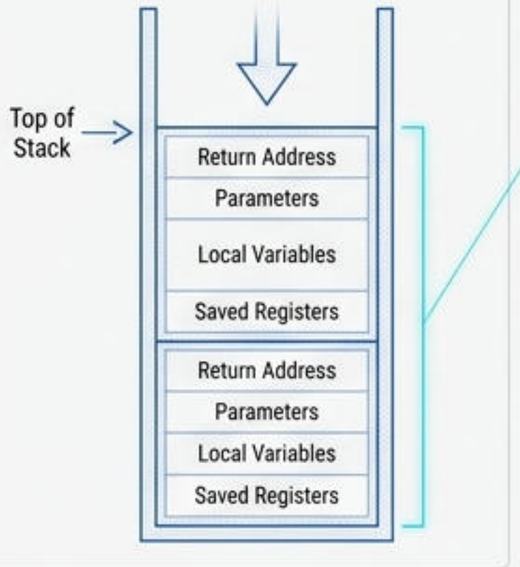
2. הכאוס: תנאי מרוץ ומניעה הדדית.

3. המנצח: אלגוריתמי תזמון מעבד.

4. העולם האמיתי: בעיות קלאסיות וניתוח אקדמי.

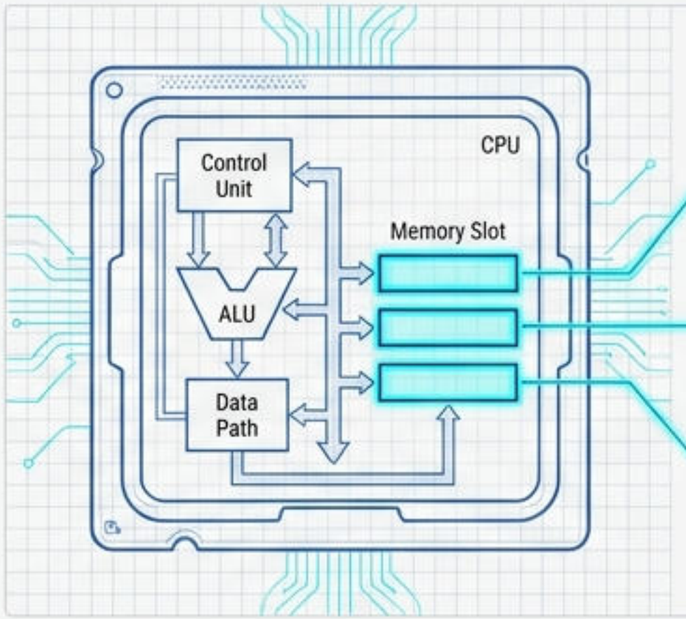
היסודות הפיזיים: אוגרים, שגרות ופסיקות חומרה

קריאה לשגרות (Function Calls)



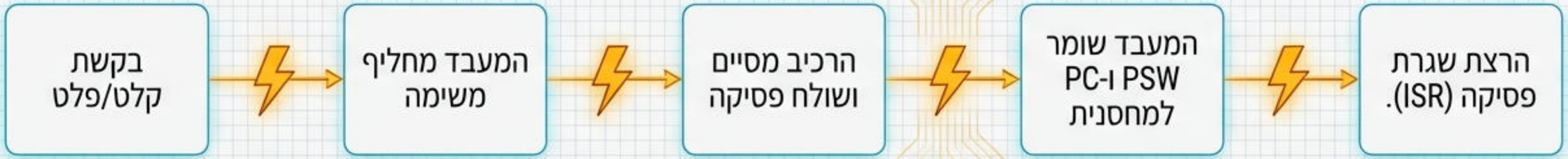
המערכת משתמשת במחסנית (Stack). כל קריאה יוצרת מסגרת (Frame) השומרת פרמטרים ואת ה-PC הישן לחזרה.

אוגרי בקרה (Control Registers)



- **PC** (Program Counter): שומר את כתובת פקודת המכונה הבאה.
- **IR** (Instruction Register): מחזיק את הפקודה המפוענחת כעת.
- **PSW** (Program Status Word): אוגר קריטי השומר את מצב המעבד.

מנגנון פסיקות חומרה (Interrupts)



מהתוכנית הסטטית אל התהליך החי (מודל 3 המצבים)

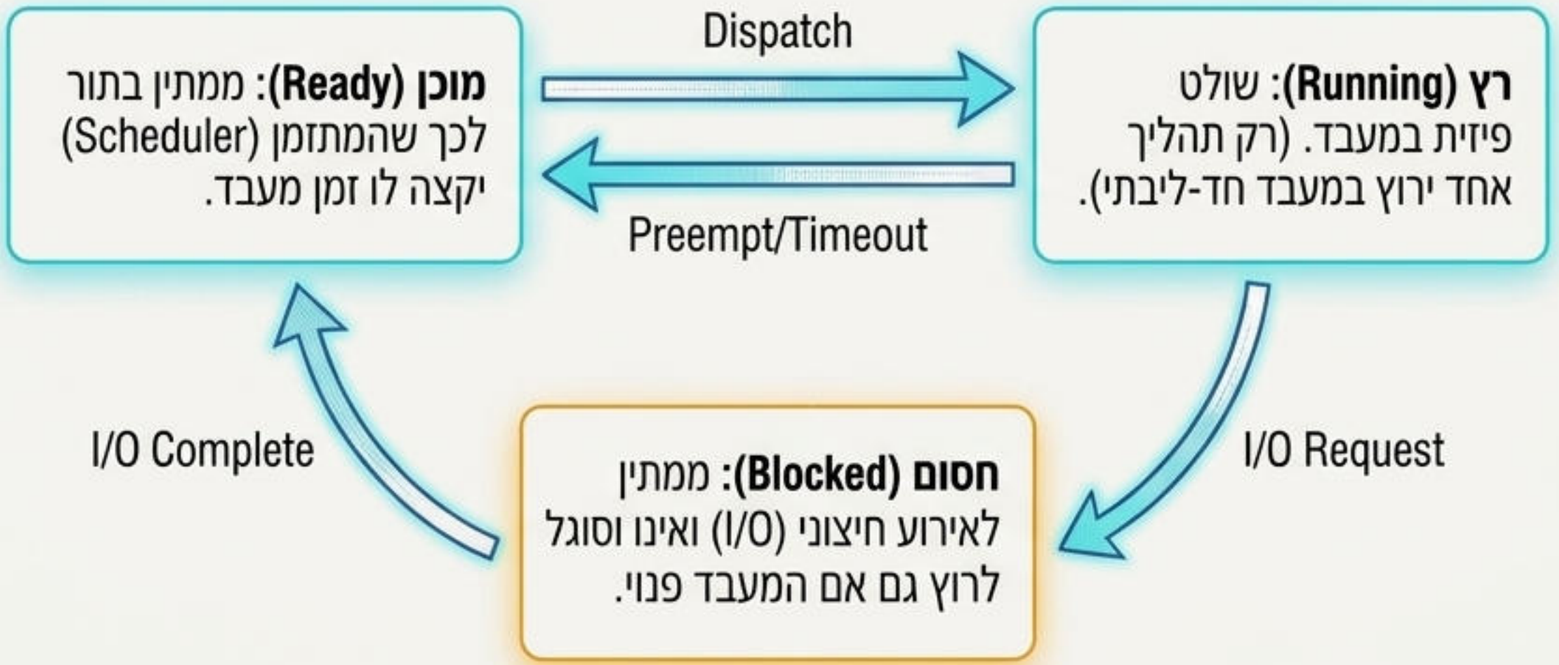


תוכנית
(קוד סטטי בדיסק)

מעבד

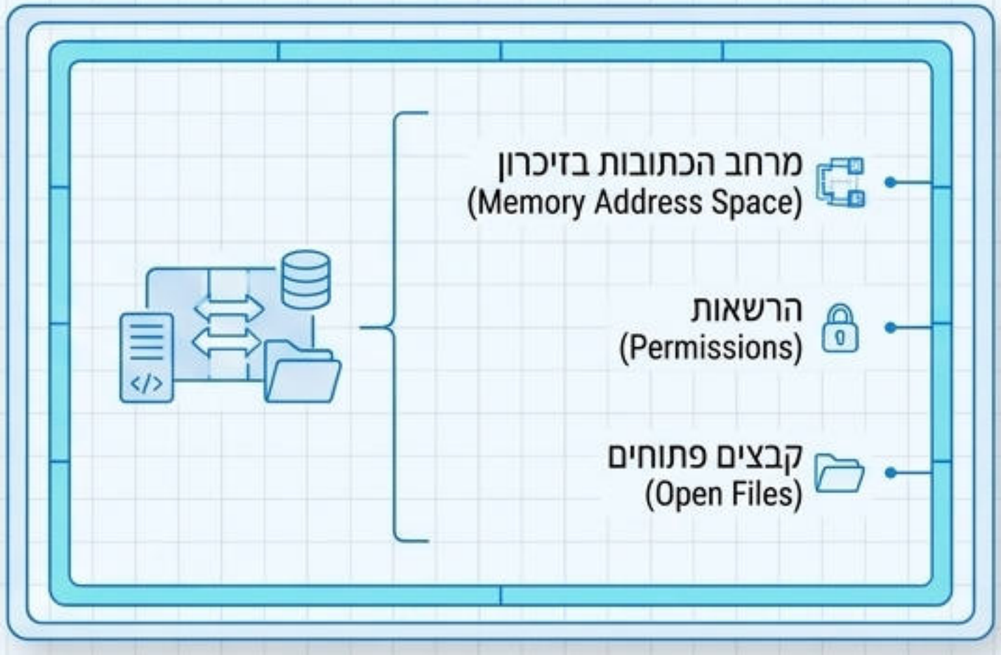
תהליך
(תוכנית בריצה)

מקביליות מדומה (Pseudoparallelism):
המעבד מחליף תהליכים במהירות עצומה ליצירת אשליית ריצה מקבילית.

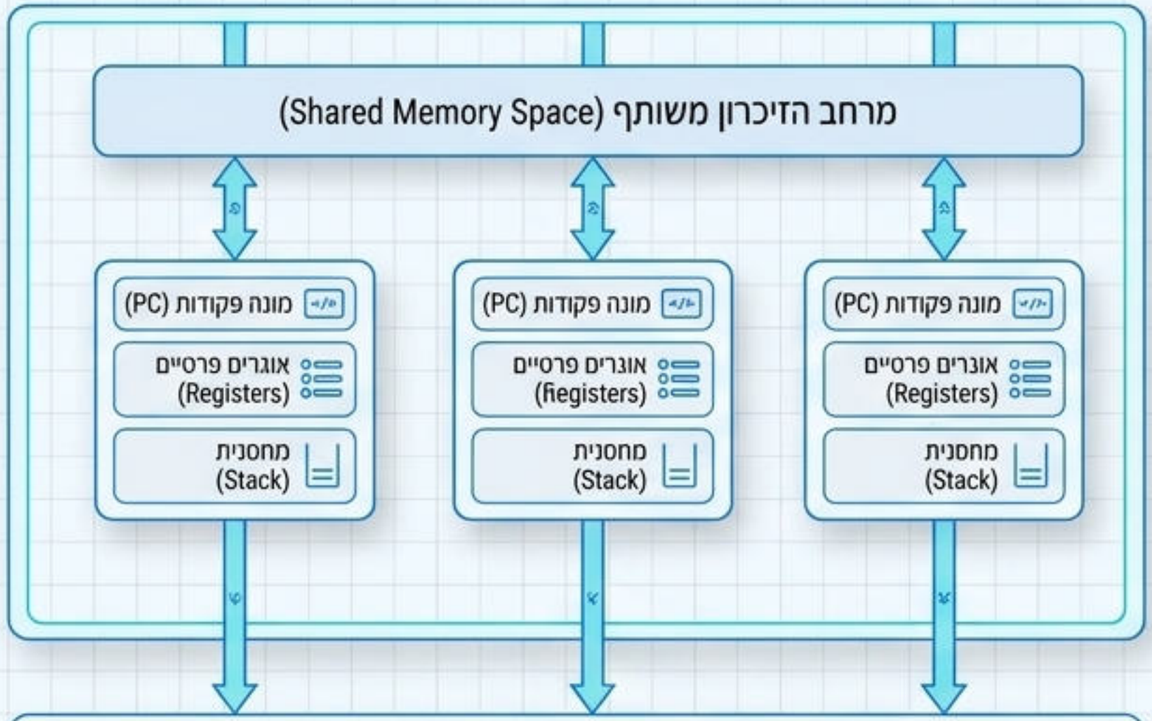


תהליכים מול תהליכונים (The Paradigm Shift)

התהליך (המעטפת): יחידת הקצאת משאבים. מחזיק את מרחב הכתובות בזיכרון, הרשאות וקבצים פתוחים.



התהליכון (חוט הביצוע): יחידת ההרצה. מחזיק מחסנית (Stack), מונה פקודות (PC) ואוגרים פרטיים. מרחב הזיכרון משותף לכל התהליכונים בתהליך.



למה תהליכונים?

- **משקל קל:** יצירה ו-Context Switch מהירים.
- **מקביליות באפליקציה:** למשל, בדפדפן - רנדור, הורדה וממשק במקביל.
- **תקשורת ישירה:** שיתוף זיכרון מייתר מנגנוני IPC כבדים.

אתגרי מקביליות ו-POSIX Pthreads

- **API מרכזי:** pthread_create, pthread_exit, pthread_exit, pthread_join pthread_yield.
- **הסכנה:** משתנים גלובליים (כמו errno) נדרסים ע"י תהליכונים אחרים.
  הפתרון: זיכרון מקומי (Thread-Local Storage) ופונקציות Thread-Safe.

היכן מנוהלים התהליכונים? (Implementation Matrix)

רמת משתמש (User-Space)

מנוהל ע"י ספריית זמן-ריצה בתוך התהליך. מערכת ההפעלה ההפעלה אינה מודעת אליהם.

✅ **מהירות עצומה:** החלפת הקשר ללא קריאות מערכת וללא פסיקות.

❌ **סכנת חסימה קטלנית:** פעולת I/O חוסמת אחת של תהליכון, תחסום את התהליך כולו.

❌ **אין תמיכה בריבוי ליבות.**

רמת גרעין (Kernel-Space)

מנוהל ישירות ע"י ה-Kernel.

✅ **עצמאות:** תהליכון נחסם אינו חוסם את חבריו. ניצול ריבוי ליבות מלא.

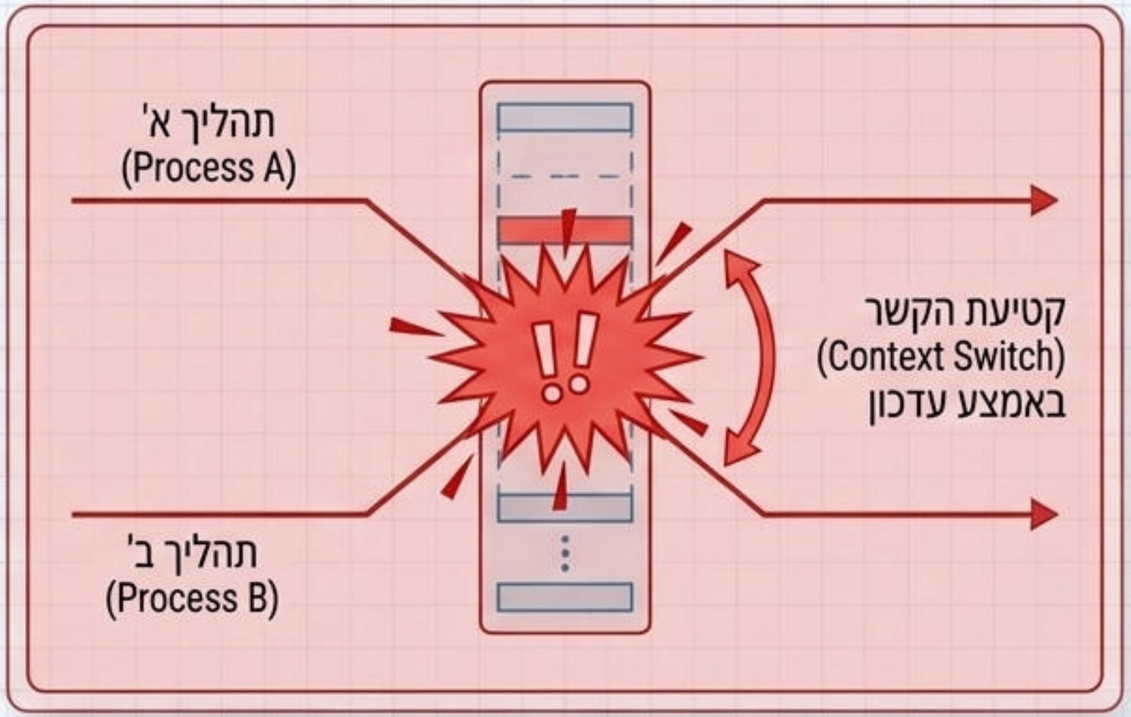
❌ **תקורה** (Overhead) **גבוהה:** כל יצירה והחלפה דורשת מעבר יקר למרחב הליבה.

מודל היברידי ותהליכונים קופצים

הפעלת מתזמן (Scheduler): (Scheduler Activations)
הגרעין מתריע למרחב המשתמש (Upcall) על חסימה, ומאפשר חסימה, ומאפשר לספרייה לשבץ תהליכון אחר על מעבד וירטואלי.

תהליכונים קופצים (Threads Pop-up Threads): לשרתים מהירים. יצירת תהליכון ייעודי וריק מהיסטוריה עבור כל בקשה נכנסת, המושמד מיד בסיום הטיפול.

כאוס במקביליות: תנאי מרוץ ומניעה הדדית



תנאי מרוץ (Race Condition): מצב שבו מספר תהליכים ניגשים לזיכרון משותף, והתוצאה תלויה בסדר הורצה האקראי של המעבד. קטיעת תהליך באמצע עדכון תגרום לדריסת נתונים.

קטע קריטי (Critical Region): קטע הקוד שבו מתבצעת הגישה לזיכרון המשותף.

4 כללי הזהב למניעה הדדית (Mutual Exclusion):



1. מניעה הדדית מוחלטת:
לעולם לא ישהו שני תהליכים תהליכים בקטע הקריטי יחד.



2. אי-תלות בחומרה:
הפתרון אינו תלוי במהירות המעבד או במספר הליבות.



3. אי-חסימה מבחוץ:
תהליך מחוץ לקטע הקריטי לא ימנע מאחרים להיכנס אליו.



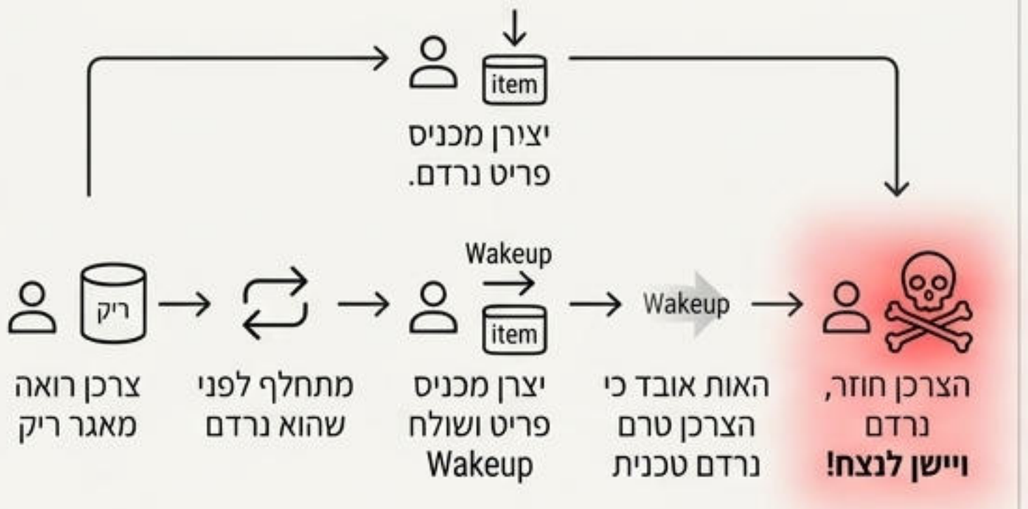
4. ללא הרעבה (Starvation No): תהליך יזכה להיכנס בסופו של דבר.

האבולוציה של הסנכרון: מהמתנה פעילה לחסימה

חסימה (Blocking)

הפסקת המתנה פעילה באמצעות Sleep / Wakeup.

הבאג הקריטי: סכנת ההשכמה האבודה (Lost Wakeup Problem)



המתנה פעילה (Busy Waiting / Spinlock)

1. **השבתת פסיקות:** מסוכן חמ ולא עובד בריבוי ליבות.
2. **משתני תור** (Strict Alternation). כופה תורנות קשיחה.
3. **אלגוריתם פטרסון: שילוב** שילוב מערך כוונות ומשתנה תור.
4. **פקודת חומרה** (TSL): פקודה אטומית הנועלת משתנה פיזית.

ארגז הכלים לסנכרון: סמפורים ומעבר



סמפור (Semaphore - Dijkstra 1965)

מונה אטומי שומר-זיכרון הסופר השכמות עתידיות ומונע השכמה אבודה.

Down (Wait / P): בודק מונה. אם > 0, מוריד ב-1. אם 0, חוסם את התהליך (מרדים ללא המתנה פעילה).

Up (Signal / V): מגדיל ב-1. אם יש ממתינים, מעיר אחד.

קוד יצרן-צרכן מבוסס סמפורים:

```
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

// יצרן
down(&empty);
down(&mutex);
insert(item);
up(&mutex);
up(&full);

// צרכן
down(&full);
down(&mutex);
item = remove();
up(&mutex);
up(&empty);
```

כלים מתקדמים:



Mutex: סמפור בינארי (0/1). יעיל לנעילה פשוטה.



Futex: היברידי. נעילה אטומית במרחב משתמש.



Monitors: פתרון בשפה עילית (למשל Java). הקומפיילר מנהל את ה-Mutex אוטומטית.



Message Passing: תקשורת ללא זיכרון משותף.



Barriers (מחסומים): סנכרון שלבי של קבוצה.



הורשת עדיפויות (Priority Inheritance): מניעת היפוך עדיפויות.

דילמת המתזמן: מטרות לפי סביבת עבודה

רמות התזמון:

1. **טווח קצר (CPU):** מחליט מי יעלה מיד מ-Ready ל-Running.
2. **טווח בינוני (Swap):** מנהל זיכרון. מי יפונה לדיסק עקב עומס ב-RAM.
3. **טווח ארוך (Admission):** מחליט האם לאפשר למשימה חדשה להיכנס בכלל למערכת.

מערכות אצווה (Batch)

המטרה: תפוקה (Throughput) **וניצולת** מקסימלית.

מערכות אינטראקטיביות (Desktop/Web)

המטרה: זמן תגובה ומידתיות (בקשה פשוטה = תגובה מהירה).

מערכות זמן-אמת (Real-Time)

המטרה: חיזוי, דטרמיניזם **ועמידה ב-**Deadlines (קשיח מול רך).

הפרדת רשויות בארכיטקטורה (Mechanism vs. Policy)

הגרעין קובע איך מבצעים Context Switch (המנגנון), אך אפליקציה במרחב המשתמו לקבוע את אלגוריתם התעדוף (המדיניות).

השוואת אלגוריתמי תזמון (Scheduling Algorithms)

אלגוריתמים למערכות אצווה (Batch)

SRTN



הגרסה המופקעת (Preemptive) של SJF.
משימה קצרה חדשה מפקיעה מעבד מיד.

SJF (Shortest Job First)



האופטימלי מתמטית.
חסרון: דורש נבואת העתיד.

FCFS (First-Come, First-Served)



הראשון מקבל.
סכנה: אפקט השיירה (Convoy Effect)

אלגוריתמים למערכות אינטראקטיביות

SPN



מיצוע מעריכי לחיזוי זמן
הריצה הבא.

(תעדוף) Priority



חשיבות לכל תהליך.
סכנה: הרעבה.
פתרון: הזדקנות (Aging).

Round Robin (RR)



חלוקת חלון זמן (קוואנטה) שווה
זמן קצר = תגובה
תגובה מהירה אך תקורה גבוהה.

MFQ (תורים היררכיים)



תהליכים עתירי חישוב יורדים רמה, תהליכי I/O
נשארים למעלה לתגובה מיידית.

(הגרלה) Lottery



הסתברותי למניעת הרעבה.
מחלקים כרטיסים.

בעיות סנכרון קלאסיות (ניתוח ופתרון)

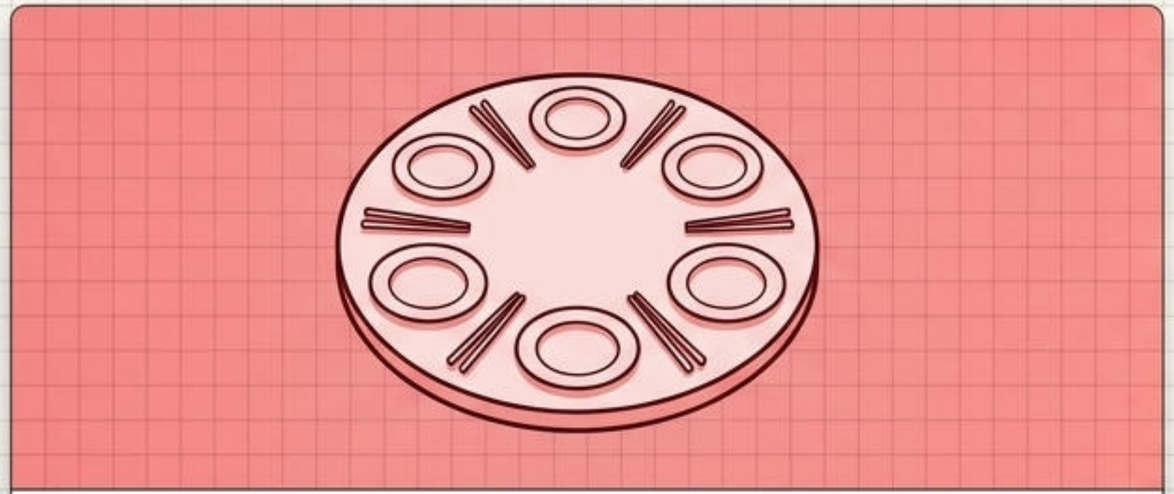


בעיית הקוראים והכותבים (Readers-Writers)

המודל: אינסוף קוראים יכולים לגשת למסד הנתונים יחד. כותב דורש גישה בלעדית לחלוטין (משאב יחיד משותף).

הסכנה - הרעבה (Starvation):

- העדפת קוראים: זרם רציף של קוראים ירעיב כותבים לנצח בחוץ.
- העדפת כותבים: זרם כותבים ינעל את המסד לקריאה.



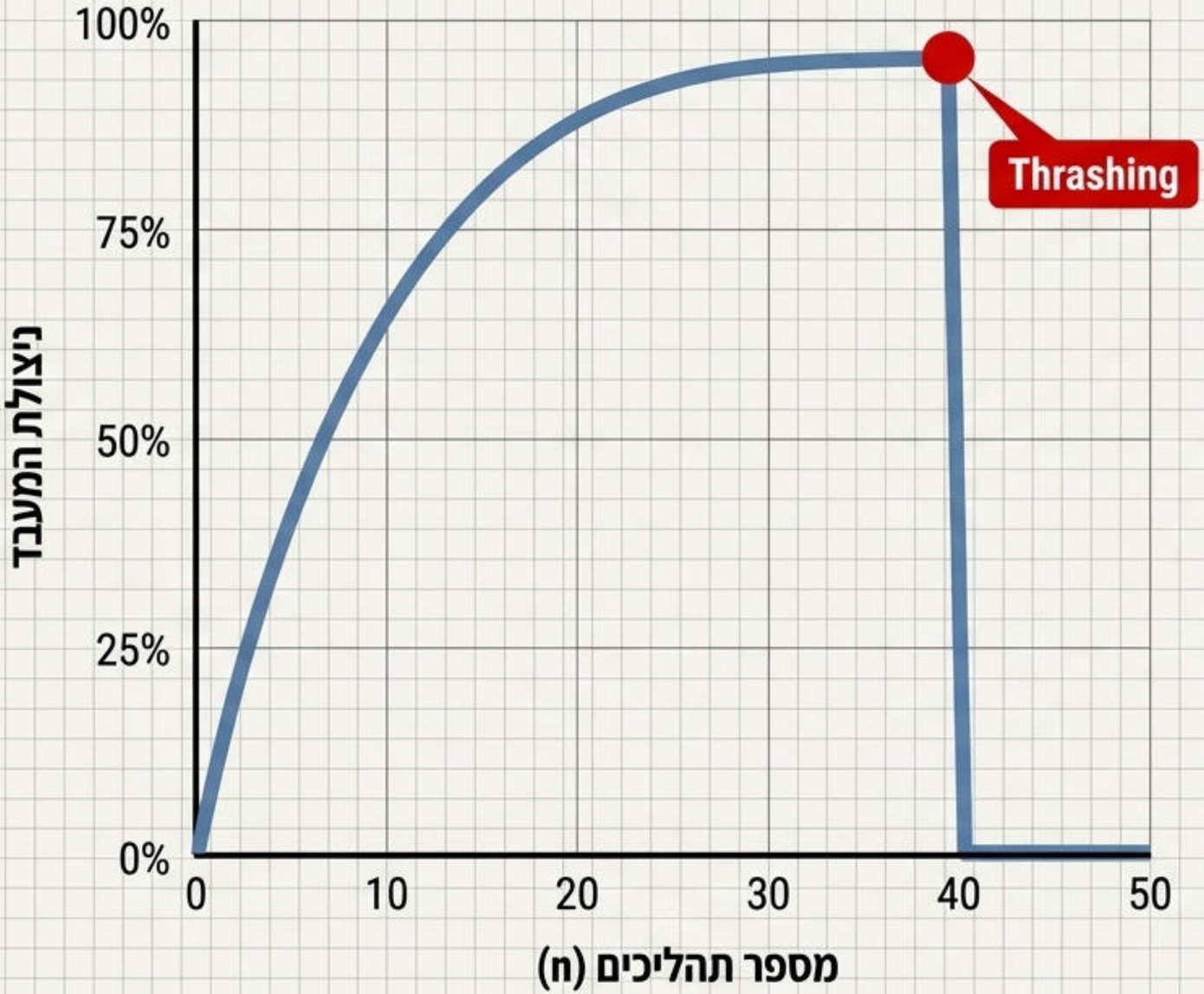
בעיית הפילוסופים הסועדים (Dining Philosophers)

המודל: 5 פילוסופים צריכים לתפוס 2 מקלות סמוכים בו זמנית כדי לאכול.

הסכנה - קיפאון מעגלי (Deadlock): כולם מרימים את המקל השמאלי יחד. אין מקל ימני פנוי, כולם ימתינו לנצח בקיפאון מוחלט.

פתרונות: אסימטרי (פילוסוף אחד מרים קודם ימין), או מערך מצבים (State Array).

מודל לריבוי תוכניות (Multiprogramming & Thrashing)



המטרה: החזקת תהליכים רבים בזיכרון, כך שחסימת I/O של אחד תנוצל לריצת אחר.

המתמטיקה של ניצולת המעבד:

אם תהליך מבלה אחוז p מזמנו בהמתנה ל-I/O, ויש n תהליכים במקביל:

$$\text{ניצולת המעבד} = 1 - p^n$$

ככל שנוסיף תהליכים (n גדל), הניצולת מתקרבת ל-100%.

נקודת השבירה (Thrashing):

כאשר הזיכרון הראשי מתמלא אל מעבר לקיבולת שלו, הניצולת קורסת לאפס.
מערכת ההפעלה נאלצת להעתיק זיכרון אל הדיסק הקשיח (Paging/Swapping).
ניהול הזיכרון חונק את המערכת והמעבד מושבת.

ניתוח אקדמי ופתרון בעיות (חלק א')

שאלה 1: ניצולת דיסקים וניהול תזמון

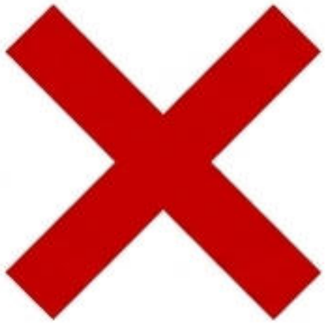
- **נתונים:** 8 מעבדים, 20 דיסקים.
- דרושים 4 דיסקים למשימה (3 רציף, 1 בסיום).

הקצאה נאיבית (4 מראש): 20 חלקי 4 = 5 משימות במקביל.
עד 5 דיסקים מושבתים בהמתנה.

הקצאה חכמה (3 + 1 דינמי): 20 חלקי 3 = 6 משימות במקביל.
מקסימום 2 דיסקים מושבתים, הניצולת עולה.

שאלה 2: לולאה אינסופית והרעבה

האם תהליך השוהה בלולאה אינסופית נמצא במצב של הרעבה (Starvation)?



תשובה: לא!

הרעבה היא מצב שבו תהליך כשיר לרוץ, אך מערכת ההפעלה מונעת ממנו גישה.
לולאה אינסופית צורכת זמן מעבד; הבעיה נובעת **מפגם לוגי בקוד**, לא ממניעת משאבים.

ניתוח אקדמי ופתרון בעיות (חלק ב')

שקילות חישובית (סמפורים מול העברת הודעות)

ניתן לממש כל מנגנון באמצעות השני, שניהם שקולים בעוצמתם.
מימוש הודעות עמי סמפורים, או תהליך סנכרון המדמה סמפור עם הודעות.

מתמטיקת עדיפויות דינמית

בונוס המתנה = α. קנס/בונוס ריצה = β.

- כאשר 0 < α > 0: בונוס הריצה גדול יותר. האלגוריתם מדרדר ל-FCFS.
- כאשר הערכים שליליים: דועך במהירות. תהליכים טריים מפקיעים מיד ויוצרים **הרעבה** של ישנים (LIFO / SRTN הפוך).

קשרי הכלה היררכיים באלגוריתמים

FCFS שייך ל-RR (כאשר הקוואנטה שואפת לאינסוף).
SJF שייך ל-Priority (כאשר העדיפות הופכית לזמן הריצה המשוער 1/T).

מפת מושגים מסכמת (The Blueprint Glossary)

יחידות ביצוע (The Entities)

תהליך (Process): תוכנית בריצה, מאגדת משאבים וזיכרון.

תהליכון (Thread): חוט ביצוע קל משקל; מחזיק PC ומחסנית פרטיים אך חולק זיכרון.

החלפת הקשר (Context Switch): פעולת הגרעין לשמירת אוגרי התהליך היוצא וטעינת הנכנס.

סכנות וכאוס (The Dangers)

תנאי מרוץ (Race Condition): התוצאה שגויה עקב תלות בסדר הרצה אקראי.

קטע קריטי (Critical Region): קטע קוד הניגש לזיכרון משותף החשוף לדריסה.

קיפאון (Deadlock): תהליכים חוסמים זה את זה במעגל ללא מוצא.

הרעבה (Starvation): תהליך כשיר נדחה כרונית עקב מדיניות מערכתית.

המתנה פעילה (Busy Waiting): סריקה רציפה בלולאה המבזבזת זמן מעבד.

השכמה אבודה (Lost Wakeup): אות חילוץ אובד כי נשלח לפני הזרדמות הטכנית.

ארכיטקטורה ופתרונות (The Solutions)

מניעה הדדית (Mutual Exclusion): דרישה שרק תהליך בודד ישהה בקטע הקריטי.

סמפור (Semaphore): מונה אטומי שומר זיכרון החוסם/מעיר ללא בזבוז מעבד.

מנעול (Mutex): סמפור בינארי להגנה בלעדית פשוטה.

מוניטור (Monitor): מבנה שפה עילית המבטיח נעילה אוטומטית.